

Now consider the C program called *kerala.c* given below:

```
int main()
{
    int Oo,0,o;
    for(O=0, o=0; Oo="DEEPU-OSFY-
=4z6{7y9v;u:v:v;t=s?r?qElFjGjHhGiB"
    "nBoDkFjC64gHhHg FiFjIgJeIgGiFj
DlDkD76c33C48cLc"
    "Kd33Jd33Hf33Gf33Gf
43Eg43Fg35HcLcKeIhFjDkCmAnC"
    "nBn?r=s>s<t9x6z3?"[O+++ 'd'-'Y'];)
while(Oo-- > 'n'-'<') putchar(++o ==
'K'? (o=0,10):33^O&1);
    return 0;
}
```

The program is definitely not self-explanatory. Figure 4 shows the output of the C program *kerala.c*. You may not be familiar with the map of Kerala, an Indian state. But a cursory search on the Internet will confirm that the ASCII art map drawn by the program is indeed that of Kerala. ASCII art is the graphic design technique that uses a computer for presentation and consists of images drawn with characters defined by ASCII. I was inspired by a similar program, which prints the ASCII art map of India. I really wanted to include that program in this article. But that program is so popular that it is used by a lot of people, and I was unable to identify the original source. I believe using that program without proper citations in this article is an injustice to the ingenious programmer who wrote it originally. But if you search the Internet with the phrase 'C program to print the map of India', you will get the program and some nice explanations. Now, coming back to our program *kerala.c*, how does it work?

Let us try to understand the working of the program. The line of code `int Oo,O,o;` declares three variables, *Oo*, *O* and *o*. The use of such variable names will confuse casual observers a lot. The core of the program is the string:

DEEPU-OSFY-=4z6{7y9v;u:v:v;t=s?r?qElFjGjH

hGiBnBoDkFjC64gHhHgFiFjIgJeIgGiFjDlDkD7
6c33C48cLcKd33Jd33Hf33Gf33Gf43Eg43Fg35H
cLcKeIhFjDkCmAnCnBn?r=s>s<t9x6z3?

The string is divided into a number of lines for readability. This string is an example of run length encoding, a lossless data compression technique. For example, the string *AAAAAAAAABBBB* can be written as *A8B4* in run length encoding because there are eight As and four Bs in the string. The characters in the string in our program tell the compiler to print spaces and exclamation marks alternatively. The characters of the string from the 0th position to the tenth position are not part of the image being drawn and hence ignored. The 11th character is `=`, whose ASCII value is 61. The program is written in such a way that 11 (61-50) spaces will be printed. The 12th character is `4`, whose ASCII value is 52 and, hence, the program prints 2 (52-50) exclamation marks. The ASCII value of the character in the odd position specifies the number of spaces to be printed, and the character in the even position specifies the number of exclamation marks to be printed. The *while* loop in the program iterates through the string, and prints a newline character or a space or an exclamation mark with the line of code `putchar(++o == 'K'? (o=0,10):33^O&1)`. The variable *O* contains the index of the character in the string being processed. If *O* contains an odd number, *O&1* becomes 1 and `33^O&1` becomes 32 (refer to the truth table for XOR operation), the ASCII value of space. On the other hand, if *O* contains an even number, *O&1* becomes 0 and `33^O&1` becomes 33, the ASCII value of exclamation mark. Thus, spaces and exclamation marks are printed in an alternating fashion. Replace the code fragment `putchar(++o == 'K'? (o=0,10):33^O&1);` with `putchar(++o == 'K'? (o=0,10):32^O&1);` to obtain the program *kerala2.c*, whose output is shown in Figure 5. Can you explain

```
File Edit View Search Terminal Help
# gcc -w ob2.c
# ./a.out
ENTER NO:7
FACTORIAL IS:5040 #
#
```

Figure 3: Output of the program *ob2.c*

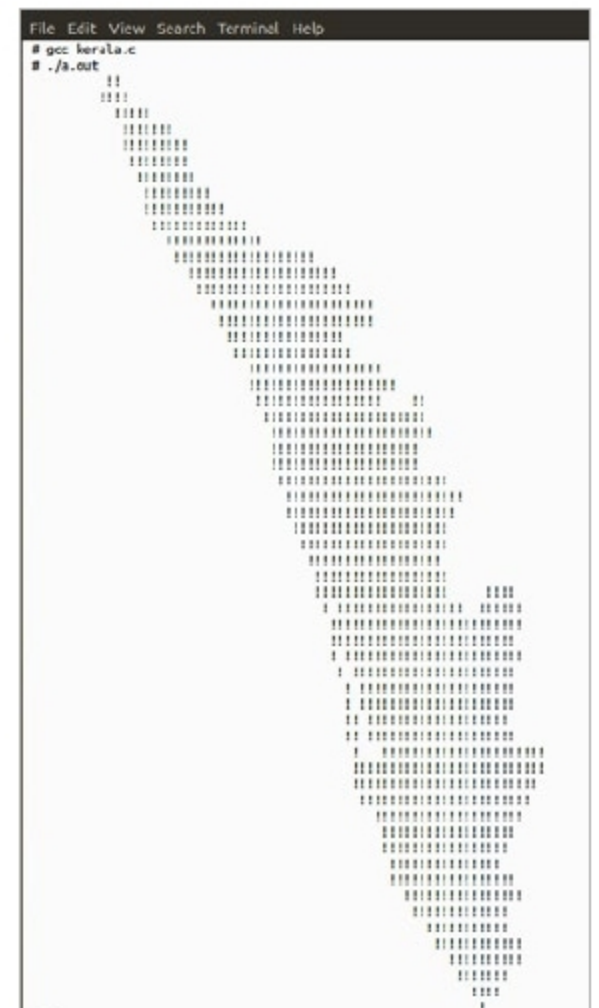


Figure 4: Output of the program *kerala.c*

the reason for the output of the program *kerala2.c*. If you haven't got the answer please download the program *kerala2.c* from the link mentioned earlier. I have added the answer as a comment. What happens if the code fragment `putchar(++o == 'K'? (o=0,10):34^O&1);` is used to replace the original code?

Yet another question that needs to be answered is, how did I get this string? Well, this is the part where my patience helped me. First, I drew the ASCII art map of Kerala by hand, referring to a map, and then wrote another program to count the number of lines, spaces and exclamation marks, and then produced the string used in the program *kerala.c*.

Now the next important question to be answered is: which programming language is suitable for code obfuscation?

Continued on page no. 94....